

Analysis Report - PCD Assignment 2

Sanindra Masya Anindita - 554741

This report documents the implementation and testing of several image enhancement techniques, covering both spatial domain and frequency domain approaches. All operations were implemented manually using NumPy, without relying on high-level image processing libraries (e.g., OpenCV's built-in filter functions), in order to demonstrate direct understanding of the underlying mathematics. The only exception is the Fast Fourier Transform (FFT), for which NumPy's built-in `np.fft.fft2/np.fft.ifft2` was used as the well-established $O(n \log n)$ transform, consistent with standard practice (naive DFT computation is rarely used in real applications due to its $O(n^2)$ complexity).

Enhancement techniques were tested against images with deliberately introduced degradations such as darkness, low contrast, blur, and noise which were created either through photo editing software or programmatically, to evaluate each method under controlled, known conditions.

Gamma Correction

Purpose: Corrects images that are too dark (underexposed) or too bright (overexposed) by non-linearly remapping pixel intensities.

Formula:

$$g(x, y) = 255 \times \left(\frac{f(x, y)}{255} \right)^\gamma$$

Each pixel is normalized to the range $[0, 1]$, raised to the power γ , then rescaled back to $[0, 255]$

Behavior:

- $\gamma < 1 \rightarrow$ brightens the image (expands dark tones), suitable for underexposed images
- $\gamma > 1 \rightarrow$ darkens the image (compresses bright tones), suitable for overexposed images
- $\gamma = 1 \rightarrow$ no change

Observed limitation: While gamma correction with $\gamma < 1$ successfully brightened dark test images, it was observed to reduce apparent contrast in bright regions of the image. This is explained by the derivative of the power-law curve:

$$\frac{dg}{df} = \gamma \left(\frac{f}{255} \right)^{\gamma-1}$$

Near the dark end of the intensity range, this slope is steep, causing shadow detail to spread apart (increased separation). Near the bright end, the slope flattens toward γ , causing highlight tones to compress toward one another (decreased separation). This means gamma correction is fundamentally a *brightness remapping* operation, not a *contrast-preserving* one — it can

improve visibility of shadow detail at the cost of highlighting contrast. In testing, chaining gamma correction with histogram equalization recovered the contrast lost in this trade-off.

3. Histogram Equalization

Purpose: Corrects images with poor contrast by redistributing pixel intensities to use the full available range (0–255), rather than being clustered in a narrow band.

Formula:

$$S_k = \frac{cdf(k) - cdf_{min}}{(M \times N) - cdf_{min}} \times (L - 1)$$

where:

- $cdf(k) = \sum_{j=0}^k h(j)$ is the cumulative distribution function of the histogram, up to intensity level k
- $h(j)$ is the number of pixels with intensity j
- $M \times N$ is the total pixel count
- $L - 1 = 255$ for 8-bit images

Implementation steps: compute the histogram of pixel intensities → compute the cumulative sum (CDF) → normalize the CDF to the range [0, 255] → remap every pixel intensity through this lookup table.

Result: When applied to a deliberately low-contrast test image (created by compressing the output intensity range in image editing software), histogram equalization successfully stretched the compressed intensity distribution back toward the full 0–255 range, visibly restoring contrast, as confirmed by comparing histograms before and after processing.

4. 2D Convolution (Shared Filtering Engine)

Purpose: Serves as the underlying mechanism for several of the filters below (Laplacian sharpening, Gaussian blurring). Convolution replaces each pixel with a weighted sum of its neighboring pixels, where the weights are defined by a kernel.

Formula:

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] \cdot F[i - u, j - v]$$

where F is the input image, H is the kernel of size $(2k + 1) \times (2k + 1)$, and G is the output image.

Implementation notes: The kernel is flipped 180° before being applied to distinguish true *convolution* from *correlation*. The image is padded at its borders (edge-replication) so the kernel does not run out of bounds when processing pixels near the image edges.

Different enhancement effects (blurring vs. sharpening) are achieved purely by changing which kernel is passed into this shared convolution function, rather than implementing separate sliding-window logic for each effect.

5. Laplacian Sharpening

Purpose: Sharpens an image by detecting regions of rapid intensity change (edges) and amplifying them.

Kernel:

$$H = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Formula:

$$\nabla^2 f(x, y) = f(x + 1, y) + f(x - 1, y) + f(x, y + 1) + f(x, y - 1) - 4f(x, y)$$

$$g(x, y) = f(x, y) - \nabla^2 f(x, y)$$

The result is subtracted (rather than added) because the center coefficient of this particular mask is negative, following the standard sign convention for Laplacian sharpening.

6. Gaussian Blur (Internal Component)

Purpose: Used internally as a building block for unsharp masking; not intended as a standalone enhancement, since blurring reduces rather than improves perceived image clarity.

Kernel formula:

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This produces a set of weights shaped like a bell curve where the center pixel receives the highest weight, with weights decreasing smoothly with distance from the center. The parameter σ controls the spread of the blur; the kernel is normalized so its weights sum to 1, which preserves overall image brightness. The kernel is then applied via the convolution formula from Section 4.

7. Unsharp Masking

Purpose: Sharpens blurred images by identifying the detail that was lost to blurring, then adding that detail back into the image with amplified strength.

Formula:

$$detail(x, y) = f(x, y) - blurred(x, y)$$

$$g(x, y) = f(x, y) + \alpha \cdot detail(x, y)$$

where α controls sharpening strength.

Result: When tested on an image with a mild (1px radius) Gaussian blur applied, unsharp masking at default parameters ($\alpha = 1.5$) produced only a marginal, difficult-to-perceive improvement. This is expected, since a 1px blur removes very little high-frequency detail, leaving little "lost detail" to recover. Increasing the sharpening strength ($\alpha = 4.0$, larger kernel) produced a clearly visible improvement in edge definition (glasses, hair, clothing texture), confirming the function operates correctly, but at the cost of amplifying visible grain/noise in low-detail background regions. This illustrates a known limitation of unsharp masking: it amplifies *all* high-frequency content indiscriminately, including sensor noise, since it cannot distinguish genuine edges from noise.

8. Gaussian Deblurring (Wiener Deconvolution)

Purpose: Attempts to mathematically *reverse* an actual Gaussian blur (as opposed to sharpening, which only boosts existing edge contrast without recovering true lost information), using frequency-domain deconvolution.

Formula:

$$\hat{F}(u, v) = \left[\frac{H^*(u, v)}{|H(u, v)|^2 + K} \right] \cdot G(u, v)$$

where:

- $G(u, v) = F\{blurredimage\}$, the FFT of the blurred input
- $H(u, v) = F\{PSF\}$, the FFT of the assumed Gaussian point spread function
- $H^*(u, v)$ is the complex conjugate of $H(u, v)$
- K is a regularization constant controlling sensitivity to noise
- $\hat{F}(u, v)$ is the estimated FFT of the original, unblurred image

The restored image is recovered via the inverse FFT:

$$\hat{f}(x, y) = F^{-1}\{\hat{F}(u, v)\}$$

This directly follows the frequency-domain filtering pipeline: transform to frequency domain → apply a filter → transform back to the spatial domain.

Result: When tested on an image with a known, deliberately-applied Gaussian blur, Wiener deconvolution (with σ approximately matched to the blur radius) produced a visibly sharper result than unsharp masking on the same image, without the noise amplification observed in the unsharp-masked version. This supports the theoretical expectation that deconvolution, being mathematically designed to invert a specific known degradation, outperforms general-purpose edge-contrast sharpening when the degradation matches the assumed blur model. However, this method's effectiveness depends on how closely the assumed kernel (σ) matches the true blur, a parameter sweep across several σ values was used to identify the closest match, since the true blur parameters are not directly known when working from an arbitrary photo editor's "radius" setting.

Limitation: This method assumes the blur can be modeled as Gaussian. Real-world blur (motion blur, out-of-focus blur, compression artifacts) may not match this model, and results would vary accordingly. Additionally, the regularization constant K presents a trade-off: smaller values produce sharper results but amplify noise and ringing artifacts, while larger values are safer but yield less pronounced deblurring.

9. Median Filtering

Purpose: Removes salt-and-pepper (impulse) noise that are sparse, extreme-valued pixels, without the blurring side effects of linear smoothing filters.

Formula:

$$g(x, y) = \text{median}\{f(s, t) \mid (s, t) \in S_{xy}\}$$

where S_{xy} is the neighborhood window (e.g., 3×3) centered at (x, y) . Unlike the filters above, this is a *non-linear* operation which cannot be expressed as a weighted sum via convolution, since no fixed set of weights produces a median.

Result: Testing revealed that the effectiveness of median filtering is highly dependent on the *type* of noise present, not merely its presence:

- When applied to noise generated by a standard photo editor's "Add Noise" filter (likely Gaussian/uniform in distribution), median filtering produced minimal improvement and, if anything, slightly reduced fine detail (e.g., hair texture) without visibly removing the noise pattern.
- When applied to programmatically-simulated true salt-and-pepper noise (2% of pixels randomly set to pure black or white), median filtering almost completely eliminated the visible noise while preserving fine image detail, closely matching the appearance of the clean original.

This confirms the theoretical expectation: median filtering relies on a majority of unaffected pixels within each neighborhood to "outvote" a sparse corrupted outlier. Against dense, low-amplitude noise (where every pixel is affected slightly, rather than a few pixels being affected extremely), there is no clear outlier for the median to reject, and the filter instead behaves similarly to a mild blur.

10. Conclusion

This project implemented six core image enhancement functions, spanning both spatial-domain filtering (gamma correction, histogram equalization, Laplacian sharpening, unsharp masking, median filtering) and frequency-domain filtering (Wiener deconvolution). Testing against deliberately controlled degradations revealed several notable findings beyond simple "before/after" comparisons:

1. Gamma correction, while effective for brightness, reduces contrast asymmetrically due to the non-linear nature of the power-law curve, and benefits from being paired with histogram equalization.
2. Sharpening techniques (Laplacian, unsharp masking) can only amplify existing edge contrast, and cannot recover detail genuinely lost to blurring which is a distinction confirmed by comparing their results directly against Wiener deconvolution, which is mathematically designed to invert a specific blur model.
3. Median filtering's effectiveness is highly specific to impulse-type (salt-and-pepper) noise; against Gaussian-style noise, it offers little benefit and can slightly degrade fine detail.

These results demonstrate that "image enhancement" is not a single universal operation but a set of tools each suited to a specific type of degradation, and that correctly diagnosing the type of degradation present is a prerequisite for selecting an effective enhancement method.

11. Results



Original photo used

Original (Dark)



Gamma Corrected



Original (Bright)



Gamma Corrected



Original (Low Contrast)



Histogram Equalized



Original (Blurry)



Unsharp Masked



Original (Blurry)



Laplacian Sharpened



Original (Blurred)



Deblurred (Wiener)



Original (Noisy)



Median Filtered

